ELLORA DEVULAPALLY, TAFT HARRELL, DR. KATY WILLIAMS

JIT COMPILER IR VISUALIZATION

DAVIDSON COLLEGE

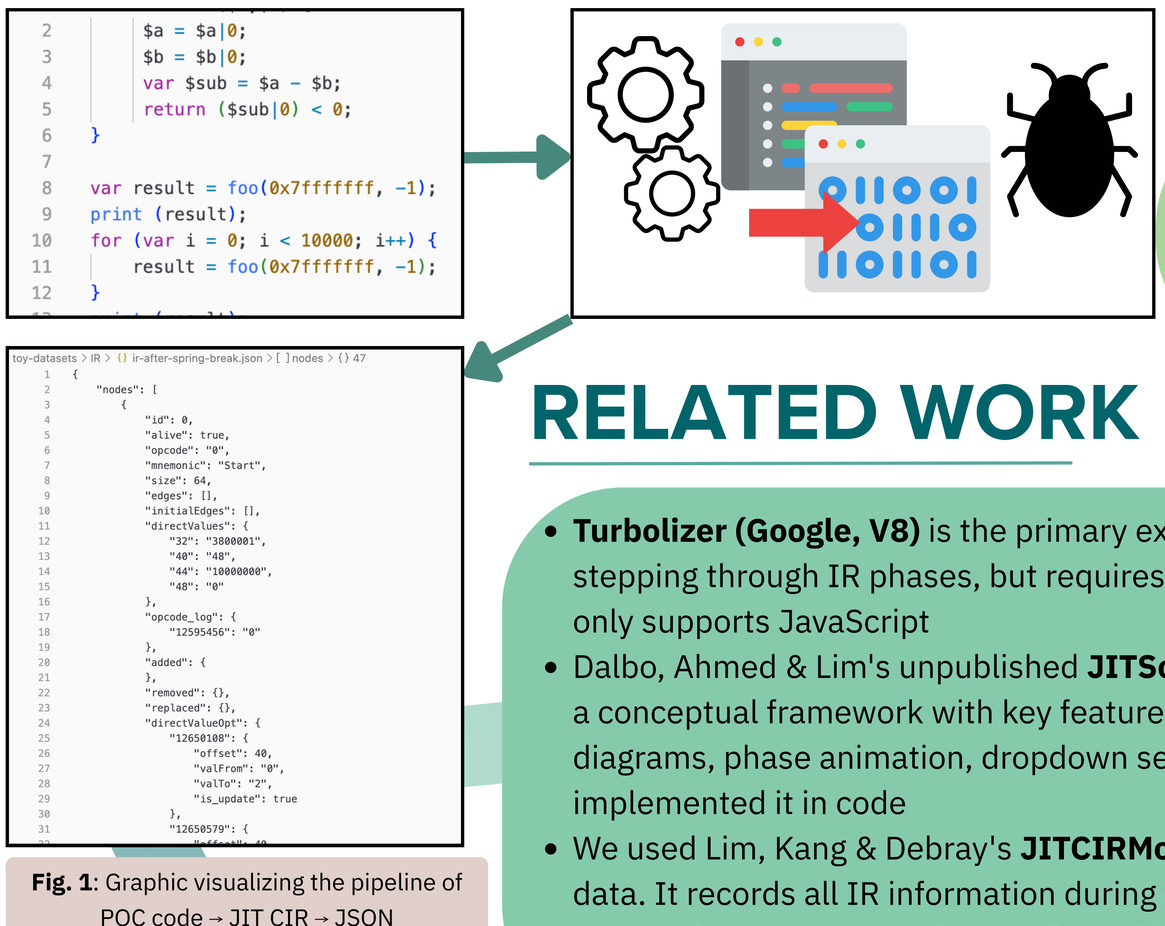


Fig. 1: Graphic visualizing the pipeline of POC code → JIT CIR → JSON

INTRODUCTION

- **Compilers** translate human-readable source code into machine code
- **Just-in-time (JIT) compilers** differ from traditional compilers by optimizing code simultaneously with execution and are used by millions on the web every day
- Compilers produce an **Intermediate Representation (IR)**: a graph of nodes (operations) and edges (relationships) that evolves across optimization phases

RELATED WORK

- **Turbolizer (Google, V8)** is the primary existing tool for stepping through IR phases, but requires complex setup and only supports JavaScript
- Dalbo, Ahmed & Lim's unpublished **JITScope** paper proposes a conceptual framework with key features (node-link diagrams, phase animation, dropdown selectors) but never implemented it in code
- We used Lim, Kang & Debray's **JITCIRModeler** to generate our data. It records all IR information during compilation and outputs it as a **structured JSON file**, which served as the foundation for our visualization

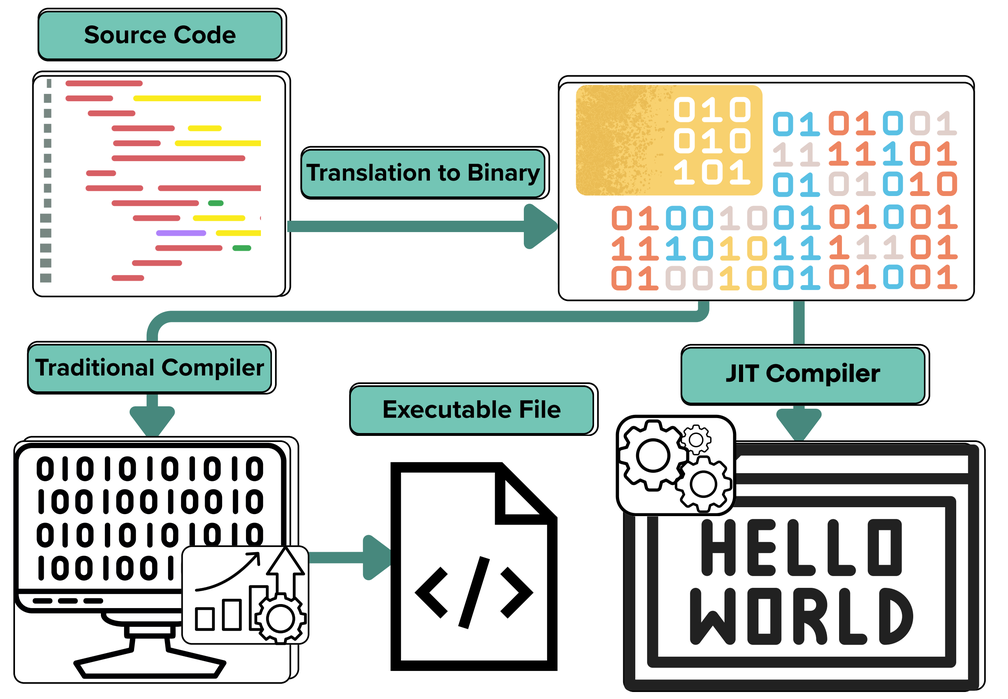


Fig. 2: Diagram outlining key differences between traditional compilers and JIT compilers

OBJECTIVES

- Build an interactive visualization that lets developers observe how a JIT compiler's IR evolves across optimization phases, making bugs faster to identify
- Create a visualization accessible to developers that is language-agnostic and easier to use

METHODOLOGY

- **Data generation**: JITCIRModeler compiled proof-of-concept file & outputed a JSON file exceeding 750,000 lines.
- **Python parsing**: We wrote Python scripts to understand the JSON's structure & each nodes' stored attributes.
- **Phase-specific data structures:** We built a JavaScript map keyed by optimization phase of [source → target] node pairs at each phase.
- **Expert consultation:** Ongoing conversations with a Davidson College compiler expert informed our understanding of the data and what the visualization would surface.
- **Iterative development & verification:** A beta static visualization was built first, then verified using a second Python script that independently reconstructed the edge list per phase from raw JSON.

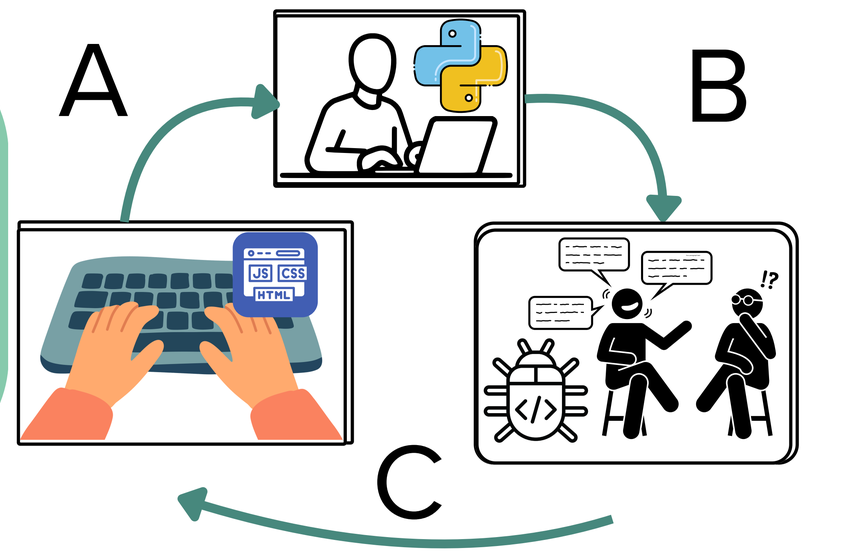
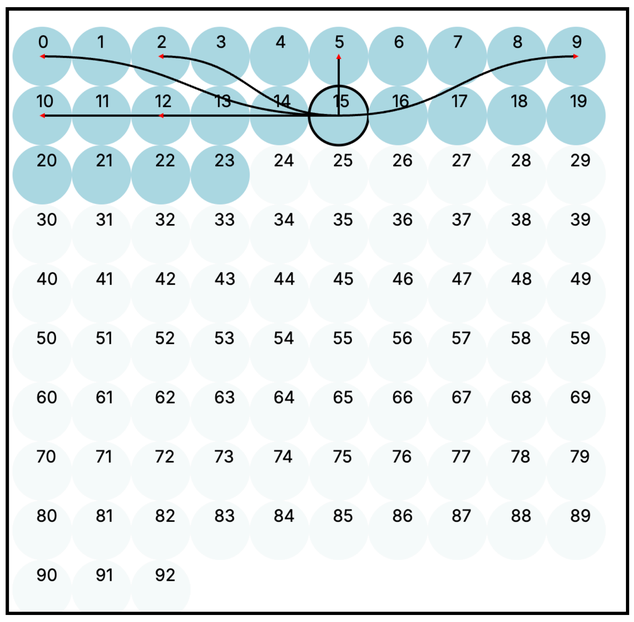


Fig 3: Python coding (**A**), expert conversations (**B**), and JS coding (**C**) iterative methodology process

RESULTS



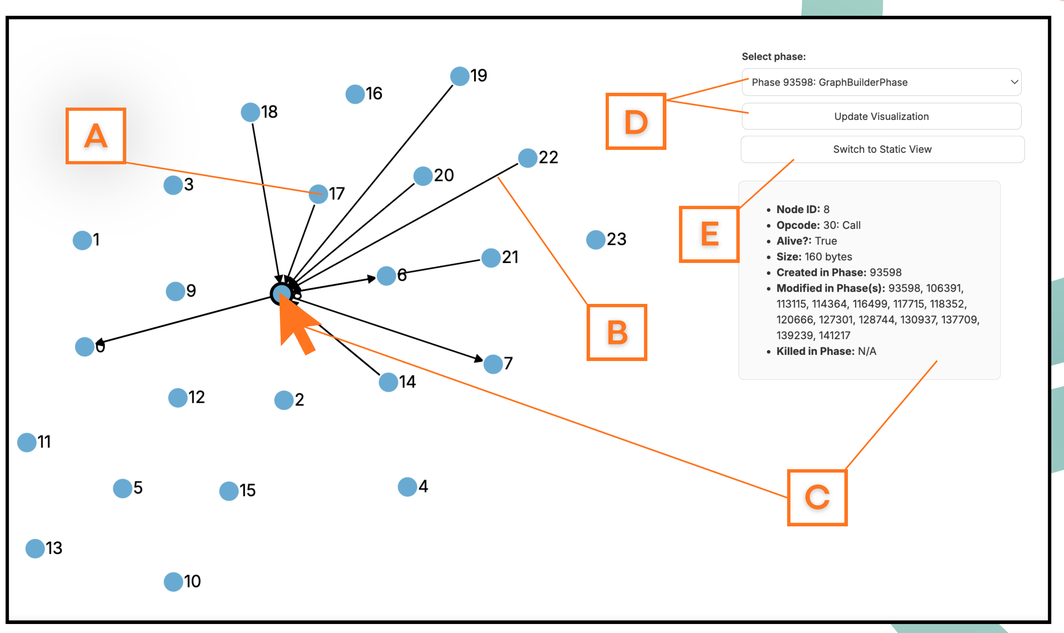


Fig 4: Static view of our prototype (left) and force-directed layout of prototype (right).

Visualization Components:

- A - **Nodes**: Only live nodes for the selected phase are shown; dead nodes are hidden
  - Static view keeps all nodes visible at all times, with dead nodes rendered at reduced opacity.
- B - **Edges**: Directional arrows show source-to-target data flow at the current phase
- C - **Tooltip**: Hover over any node to see its phase-specific operations and attributes
- D - **Phase Dropdown**: Navigate between optimization phases to track IR changes over time
- E - **Toggle**: Switch between static and force-directed layout views

CONCLUSION

- Our prototype gives developers an accessible & interactive way to trace a JIT compiler's IR evolution.
- By transforming hundreds of thousands of lines of raw JSON into a clean force-directed graph, we show that complex compiler internals can be made interpretable through careful data processing and visual design.
- Future work includes structured interviews with real-world compiler developers, user testing, and improving readability for larger IRs.

ACKNOWLEDGEMENTS

We'd like to thank Dr. Terrence Lim for his generous expert help throughout this process.